
boolean



Iterare Array in JSX

utilizzo del metodo map

Lavorare con gli Array in JSX

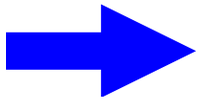
Abbiamo osservato quanto sia semplice inserire il valore di una variabile di tipo stringa o numero all'interno del markup **JSX**.

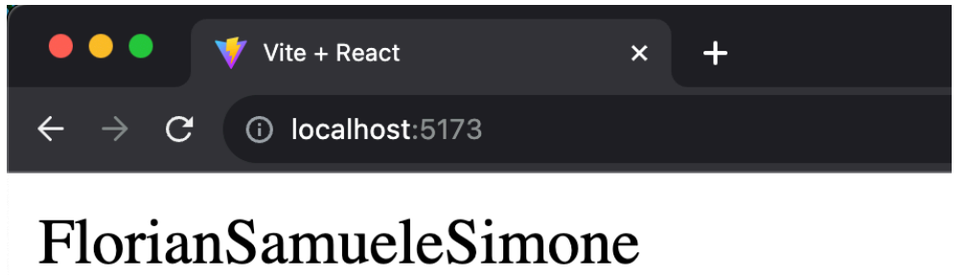
Ma come si comporta **JSX** con gli array? 🤔

Facciamo un esperimento! Proviamo a stampare all'interno di un elemento div un semplice array di stringhe e osserviamo il risultato nel browser!

```jsx
// App.jsx
const teachers = ["Florian", "Samuele", "Simone"];

export default function App() {
  return (
    <div>
      {teachers}
    </div>
  );
}
```





Lavorare con gli Array in JSX

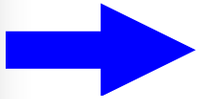
Avrete notato che gli elementi dell'array vengono stampati uno dopo l'altro. Ciò avviene perché JSX è in grado di estrarre ogni singolo elemento dalla lista e visualizzarlo all'interno della pagina.

Ma cosa dovremmo fare se volessimo che ogni elemento fosse contenuto in un altro tag, come ad esempio un list item?

Potremmo modificare il nostro array, trasformandolo da un semplice array di stringhe a un array di elementi JSX.

```jsx
// App.jsx
const teachers = [<li>Florian</li>, <li>Samuele</li>, <li>Simone</li>];

export default function App() {
  return (<ul>{teachers}</ul>);
}
```



- Florian
- Samuele
- Simone

Lavorare con gli Array in JSX

Abbiamo ottenuto il risultato desiderato, ma ora il nostro array **NON è più riutilizzabile** poiché contiene informazioni di formattazione!

Inoltre, in questo modo, diventerebbe molto difficile da modificare o aggiornare.

Viene in mente qualcosa in JavaScript che possa **restituire un NUOVO array "trasformato" senza modificare l'array di partenza?**

🤔

Iterare Array in JSX

Il metodo .map di JavaScript

Per **iterare su un array** all'interno di **JSX,** sfruttiamo il metodo **map** di JavaScript. Questo metodo crea un **nuovo array** che contiene gli elementi di partenza trasformati in base a una funzione di callback passata come argomento.

Possiamo quindi restituire un array di **componenti React** o **elementi JSX** all'interno del componente.

```jsx
// App.jsx
const teachers = ["Florian", "Samuele", "Simone"];

export default function App() {
  return (
    <ul>
      {teachers.map((teacher) => (
        <li>{teacher}</li>
      ))}
    </ul>
  );
}
```

Iterare Array in JSX

la prop key

La prop "**key**" è un attributo speciale che deve essere assegnato a ciascun elemento di una lista renderizzata da un componente.

È un **identificatore univoco** che aiuta React a tenere traccia e identificare gli elementi nella lista.

L'uso corretto della prop "key" è importante per garantire un rendering efficiente e la gestione delle modifiche in liste dinamiche.

```jsx
// App.jsx
const teachers = ["Florian", "Samuele", "Simone"];

export default function App() {
  return (
    <ul>
      {teachers.map((teacher, index) => (
        <li key={index}>{teacher}</li>
      ))}
    </ul>
  );
}
```

Se l'array è dinamico e soggetto a modifiche (aggiunta, rimozione o riordinamento degli elementi), è meglio evitare di utilizzare l'indice come chiave. In questo caso, potrebbero verificarsi problemi con la corrispondenza tra le chiavi e gli elementi reali, causando comportamenti imprevisti o errori nell'applicazione.





LIVE CODING



ESERCIZIO